

ABV-Indian Institute of Information Technology and Management Gwalior
Department of Electrical and Electronics Engineering

FPGA-Based Edge AI System for Real-Time Arrhythmia Detection

An Efficient Hybrid-Parallel Deep Learning Implementation on PYNQ-Z2

Submitted by:

Belana Biswajeet Patro

2023BEE011

bee_2023011@iiitm.ac.in

Hanima V V

2023BEE015

bee_2023015@iiitm.ac.in

Khushi Mangla

2023BEE021

bee_2023021@iiitm.ac.in

Prince Agrawal

2023BEE029

bee_2023029@iiitm.ac.in

Shejal Yadav

2023BEE031

bee_2023031@iiitm.ac.in

Submitted to: Prof. Manisha Pattanaik

Abstract

Cardiovascular diseases remain one of the leading causes of mortality worldwide, and the timely, accurate detection of cardiac arrhythmias is critical to preventing life-threatening events such as stroke and sudden cardiac death. Electrocardiography (ECG) is the clinical gold standard for diagnosing arrhythmias; however, conventional approaches rely on cloud-based, computationally intensive deep learning models that introduce high network latency and raise patient data privacy concerns, making them unsuitable for real-time wearable deployment.

This project presents an FPGA-based Edge AI system for real-time arrhythmia classification on the Xilinx PYNQ-Z2 development board. The system employs a custom lightweight Hybrid-Parallel 1D-CNN trained on the MIT-BIH Arrhythmia Database, classifying ECG beats according to the AAMI EC57 five-class scheme (N, S, V, F, Q), with Butterworth bandpass filtering (0.5–40 Hz), R-wave centred segmentation into 186-sample windows, and combined SMOTE+ENN class balancing. The model is synthesised into hardware via Vitis HLS and deployed as a Zynq PS-PL co-design. The deployed system achieves 93.20% accuracy, 60.63 ms inference latency, 16 beats/s throughput, and 1.542 W on-chip power, using 10,623 LUTs (20%), 14 DSPs (6.4%), 34.5 BRAMs (24.6%), and 19,240 flip-flops (18.1%).

Keywords: FPGA, Edge AI, Arrhythmia Detection, ECG Classification, 1D-CNN, PYNQ-Z2, AAMI EC57, SMOTE, Vitis HLS, Real-Time Inference, IoMT.

Contents

1	Introduction	1
1.1	Background and Clinical Motivation	1
1.2	Limitations of Existing Approaches	1
1.3	Project Objectives	2
2	Literature Review	3
2.1	Background	3
2.2	Paper 1 — Efficient ECG Arrhythmia Detection on FPGA (ETASR, 2025) . . .	3
2.3	Paper 2 — FPGA-Based 1D-CNN Accelerator for Real-Time Arrhythmia Clas- sification (JRTIP, 2025)	4
2.4	Comparison with Implemented Work	5
3	System Design and Methodology	6
3.1	Stage 1 — Dataset Preparation and Preprocessing	6
3.2	Stage 2 — CNN Model Design and Software Training	7
3.3	Stage 3 — FPGA Accelerator Synthesis (Vitis HLS)	9
3.4	Stage 4 — System Block Design and Integration (Vivado)	9
3.5	Stage 5 — PYNQ Deployment and Real-Time Testing	10
4	Results and Discussion	11
4.1	Implementation Stage Accuracy	11
4.2	FPGA Resource Utilisation	11
4.3	Real-Time Performance Metrics	12
4.4	Confusion Matrix Analysis	12
4.5	Comparison with Reference Works	13
5	Technical Challenges and Limitations	14
5.1	Technical Challenges Encountered	14
5.2	Limitations of the Current Implementation	15
6	Future Scope	15
7	Conclusion	17

1. Introduction

1.1 Background and Clinical Motivation

The heart's electrical activity is governed by a precisely timed conduction system. When this system malfunctions, the result is an arrhythmia — an irregular or abnormal heart rhythm that can range from benign ectopic beats to life-threatening ventricular fibrillation. According to the 2024 Heart Disease and Stroke Statistics report, cardiovascular diseases account for approximately one-third of all global deaths annually. Atrial fibrillation (AFib) alone is responsible for a significantly elevated stroke risk, as the disorganised atrial contractions it produces can trigger thrombus formation that migrates to the cerebral vasculature. Ventricular arrhythmias, characterised by wide, bizarre QRS complexes and compensatory pauses, can degenerate rapidly into ventricular fibrillation, causing sudden cardiac death within minutes if untreated.

The Electrocardiogram (ECG) is the primary, non-invasive clinical tool for diagnosing these conditions. A normal sinus rhythm (NSR) is characterised by a well-defined P-wave (atrial depolarisation), a narrow QRS complex representing ventricular depolarisation, and a T-wave of ventricular repolarisation, occurring at a resting rate of 60–100 beats per minute with a consistent 1:1 P-to-QRS ratio and stable R-R intervals. Deviations from this morphological and temporal template — absent P-waves, widened QRS complexes, prolonged QT intervals, or irregular R-R spacing — are the diagnostic signatures of specific arrhythmia classes.

Long-term ambulatory monitoring (Holter monitoring) can generate tens of thousands of heart-beat annotations from a single patient over 24–48 hours. Manual expert analysis of such volumes is both time-consuming and prone to fatigue-induced errors. This creates an urgent clinical need for automated, reliable, real-time classification systems that can accurately flag pathological beats as they occur, without requiring transmission to a remote server.

1.2 Limitations of Existing Approaches

Conventional automated arrhythmia detection systems fall into two broad categories, each with significant shortcomings for real-world deployment.

Cloud-based deep learning models: High-accuracy CNN and LSTM architectures trained on GPU clusters deliver strong software performance, but they require continuous data transmission over a network. This introduces network latency that is clinically unacceptable during acute cardiac events, raises patient data privacy and security concerns under regulations such as HIPAA

and GDPR, and creates a dependency on reliable network infrastructure that may not be available in remote or resource-constrained settings.

Traditional on-device ML models: Rule-based algorithms and classical machine learning methods such as Support Vector Machines (SVM) and Random Forests require handcrafted feature engineering that demands significant domain expertise and cannot generalise well across patient populations. More recently, software implementations of deep CNNs on microcontrollers are constrained by the sequential execution model of general-purpose processors, limiting throughput and increasing latency.

Field-Programmable Gate Arrays (FPGAs) offer a compelling alternative. Their reconfigurable fabric enables massively parallel computation, their power consumption is orders of magnitude lower than GPU-based solutions, and their deterministic execution model guarantees consistent, bounded latency — a critical requirement for safety-critical healthcare applications. However, traditional FPGA development using Hardware Description Languages (HDL) such as VHDL or Verilog is complex, time-consuming, and requires deep hardware expertise, creating a barrier to adoption in the medical AI domain.

1.3 Project Objectives

This project addresses these gaps by developing a complete hardware-software co-design pipeline that takes a custom lightweight 1D-CNN from Python-based training to real-time FPGA inference on the PYNQ-Z2 board. The PYNQ framework, built on the Xilinx Zynq-7000 SoC, provides a dual-core ARM Cortex-A9 Processing System (PS) for Python-level control and a Xilinx Artix-7 Programmable Logic (PL) fabric for hardware acceleration, bridged by a high-speed AXI interconnect.

The key technical objectives of this project are:

- To preprocess the MIT-BIH Arrhythmia Database using clinically validated techniques — Butterworth bandpass filtering, R-wave centred segmentation, and SMOTE+ENN class balancing — and map the 109,000+ raw annotations to the five standard AAMI EC57 heartbeat superclasses (N, S, V, F, Q).
- To design and train a custom Hybrid-Parallel 1D-CNN in TensorFlow that is lightweight ($\approx 25,000$ parameters), hardware-friendly, and achieves clinically relevant accuracy across all five arrhythmia classes.

- To apply model compression techniques — INT8 quantization and weight pruning — to reduce the model’s memory footprint and arithmetic intensity in preparation for FPGA deployment.
- To synthesise the compressed model into a configurable CNN accelerator IP core using Vitis HLS, leveraging `UNROLL` and `PIPELINE` pragmas to exploit the FPGA’s parallel architecture for low-latency inference.
- To integrate the CNN IP core into a complete Zynq PS-PL system in Vivado using AXI SmartConnect, generate a deployment bitstream (`.bit`) and hardware handoff file (`.hwh`), and validate the system end-to-end on the PYNQ-Z2 board via Jupyter Notebook.
- To evaluate and compare the deployed system’s accuracy, latency, power consumption, and FPGA resource utilisation against two contemporary peer-reviewed FPGA-based ECG systems from the literature.

2. Literature Review

2.1 Background

Automated arrhythmia detection from ECG signals has evolved from threshold-based rule algorithms and hand-engineered wavelet features to end-to-end deep learning models. Kiranyaz et al. [5] established that 1D-CNNs are naturally suited to ECG classification, offering lower computational complexity than 2D architectures for inherently one-dimensional temporal signals. GPU-based deep learning systems achieve high accuracy but consume 100–300 W, making them unsuitable for wearable deployment. FPGAs bridge this gap by providing reconfigurable parallel compute at low-watt power budgets and bounded, deterministic latency — both critical requirements for real-time clinical edge devices. The two papers reviewed in Sections 2.2 and 2.3 represent the current state of the art in FPGA-based ECG classification and serve as the primary benchmarks for this project.

2.2 Paper 1 — Efficient ECG Arrhythmia Detection on FPGA (ETASR, 2025)

This paper presents an FPGA-based ECG system implemented on the same Xilinx PYNQ-Z2 platform [1]. The pipeline applies a Butterworth bandpass filter (0.5–40 Hz) to MIT-BIH signals, followed by a two-level 1D Discrete Wavelet Transform (DWT) for frequency decomposition, and fiducial windowing to extract clinically meaningful features — R-R interval, wave amplitude, and R-wave slope — anchored at the R-wave peak and T-wave onset. These hand-crafted features

feed a MobileNetV2-based classifier trained in TensorFlow/Keras and deployed via the PYNQ framework. The system reports 99.7% accuracy, 0.723 s processing speed, and an ultra-low 0.42 mW power consumption.

However, these results must be interpreted carefully. The classification scheme uses four non-standard clinical categories (hyperkalemia, hypocalcemia, normal, tachycardia) rather than the AAMI EC57 five-class standard (N, S, V, F, Q), making cross-study comparison unreliable and inflating accuracy by collapsing difficult minority classes. No data augmentation or oversampling technique is reported, leaving the model exposed to the severe class imbalance in MIT-BIH (Normal class >90% of all beats). Furthermore, the 0.42 mW power figure reflects CPU-level PERF profiler measurements for a software pipeline stage, not total on-chip FPGA power, and is therefore not directly comparable to system-level power figures.

2.3 Paper 2 — FPGA-Based 1D-CNN Accelerator for Real-Time Arrhythmia Classification (JRTIP, 2025)

This paper proposes a lightweight 1D-CNN (LW-CNN) for wearable edge deployment on the Xilinx Zynq XC7Z020 [2]. The network uses seven residual blocks with Depthwise Separable Convolution (DSC) in place of standard convolution, reducing the parameter count from 22,037 to 11,736 — a 46% reduction — while maintaining accuracy. Preprocessing uses Butterworth bandpass filtering (0.5–30 Hz) and R-wave centred segmentation of 260 samples per beat. Class imbalance is addressed with SMOTE, expanding the dataset from 109,451 to 493,000 samples with equal representation across all five AAMI EC57 classes.

The model is compressed using custom unstructured pruning (>60% parameter reduction) and Incremental Network Quantization (INQ), encoding weights as powers-of-two to replace multiply-accumulate operations with bit-shifts in the FPGA accelerator — eliminating DSP block usage in the convolutional layers. The complete PS-PL co-design, synthesised via Vitis HLS and integrated in Vivado, achieves 96.55% accuracy, 63 ms latency at 50 MHz, and 1.78 W power, using only 13,726 LUTs, 9 DSPs, and 5.5 BRAMs. This is methodologically the most rigorous of the two reference works and directly motivates the HLS-based compression approach adopted in this project.

2.4 Comparison with Implemented Work

Table 1 presents a structured comparison of both reference papers against the system implemented in this project.

Table 1. Comparison of Implemented Work with Reference Papers

Parameter	Implemented Work (Our Model)	Paper 1 — ETASR 2025 [1]	Paper 2 — JRTIP 2025 [2]
Accuracy	93.20%	99.7%	96.55%
Latency / Speed	60.63 ms	0.723 s (≈ 723 ms)	63 ms
Power Consumption	1.542 W	0.42 mW	1.78 W
Platform	FPGA (PYNQ-Z2)	FPGA (PYNQ-Z2)	FPGA (Zynq XC7Z020)
Approach	Hybrid-Parallel 1D-CNN	ML + DWT + Fiducial Windowing	Lightweight 1D-CNN (LW-CNN)
Classification Standard	AAMI EC57 (5-class: N/S/V/F/Q)	Non-standard 4-class	AAMI EC57 (5-class)
Data Balancing	SMOTE + ENN (Fully Balanced)	None reported	SMOTE only
Model Compression	INT8 Quantization + Pruning	Quantization + Pruning	INQ + Unstructured Pruning
Accuracy Reliability	High — genuine multi-class AAMI evaluation	Low — 4-class, normal-class inflated	Moderate — AAMI framework, pre-SMOTE imbalance

The following observations summarise the key differentiators of the implemented system relative to both papers.

Latency: At 60.63 ms per beat, the implemented system achieves the lowest inference latency of the three, marginally ahead of Paper 2 (63 ms) and substantially faster than Paper 1 (723 ms). All three systems satisfy the real-time constraint of classifying a beat within the typical 600–1000 ms R-R interval at resting heart rates.

Accuracy and classification rigor: Paper 1’s 99.7% accuracy is obtained on a non-standard 4-class scheme without class balancing, making it an unreliable benchmark. The implemented system’s 93.20% and Paper 2’s 96.55% are both evaluated on the full AAMI EC57 5-class

scheme with proper data balancing, and are therefore more honest and clinically meaningful metrics.

Data balancing: The implemented system is the only one among the three to combine SMOTE with ENN (Edited Nearest Neighbours). SMOTE generates synthetic minority class samples, while ENN removes borderline majority class samples near class boundaries, producing a cleaner decision surface. Paper 2 uses SMOTE only; Paper 1 uses neither.

Power consumption: Paper 1’s 0.42 mW reflects only a CPU-level software stage, not total on-chip FPGA power. The implemented system’s 1.542 W and Paper 2’s 1.78 W are full-system on-chip figures measured under active FPGA inference — the correct metric for comparing edge device power budgets. Both remain orders of magnitude below GPU alternatives (100–300 W).

In summary, the implemented work achieves the best latency, the most rigorous data balancing pipeline, and full compliance with the AAMI EC57 clinical standard. Its 93.20% deployed accuracy is a more reliable and reproducible result than Paper 1’s higher but methodologically weaker figure, while its approach is directly informed by the compression and PS-PL co-design strategies demonstrated in Paper 2.

3. System Design and Methodology

The proposed system is designed as a five-stage hardware-software co-design pipeline, taking raw ECG signals from the MIT-BIH Arrhythmia Database through to real-time beat classification on the PYNQ-Z2 FPGA board. Each stage has clearly defined inputs, outputs, and tools, enabling parallel development across the team.

3.1 Stage 1 — Dataset Preparation and Preprocessing

The MIT-BIH Arrhythmia Database (PhysioNet) is the foundation of this project. It contains 48 half-hour, two-channel ambulatory ECG recordings from 47 subjects, with over 109,000 beats each individually annotated by multiple cardiologists. The raw annotations consist of dozens of beat-type labels that are mapped to the five standard AAMI EC57 superclasses prior to model training:

- Class N (Normal): Normal beats, left bundle branch block (LBBB), right bundle branch block (RBBB).
- Class S (Supraventricular): Atrial premature beats (APB), nodal escape beats.

- Class V (Ventricular): Premature ventricular contractions (PVC), ventricular escape beats.
- Class F (Fusion): Fusion of normal and ventricular beats.
- Class Q (Unknown/Paced): Paced beats, unclassifiable beats, noise.

Signal preprocessing is applied in Python on a development PC before training. A Butterworth bandpass filter with a passband of 0.5–40 Hz is applied to each raw ECG channel to simultaneously suppress baseline drift (low-frequency noise below 0.5 Hz caused by respiration and patient movement) and high-frequency interference (muscle artefacts and mains noise above 40 Hz) while preserving the clinically important P-wave, QRS complex, and T-wave morphology. Beat segmentation is performed by centring a 186-sample window around each annotated R-wave peak — 93 samples before the peak and 92 samples after — to capture a complete, normalised beat context. This input size of 186×1 was chosen to be compact enough for the on-chip BRAM while still capturing the full morphological signature of each beat type.

The raw MIT-BIH dataset is severely class-imbalanced: the Normal class (N) accounts for approximately 90% of all annotations, while classes such as Q (33 samples) and F (802 samples) are extremely under-represented. Training directly on this distribution produces classifiers biased toward Normal, with poor sensitivity for clinically critical arrhythmia classes. To address this, a two-stage data balancing pipeline is applied. First, SMOTE (Synthetic Minority Over-Sampling Technique) generates synthetic minority class samples by interpolating between existing minority instances in feature space, increasing representation of classes S, V, F, and Q. Second, ENN (Edited Nearest Neighbours) is applied after SMOTE to remove ambiguous majority class samples near class decision boundaries, producing a cleaner and more generalisable training distribution. The final balanced dataset uses an 80% / 20% train-test split, with all five classes equally represented in the training set.

3.2 Stage 2 — CNN Model Design and Software Training

Model Architecture: Hybrid-Parallel 1D-CNN

The choice of a one-dimensional CNN is justified by the inherently one-dimensional, temporal nature of ECG signals. Unlike 2D-CNNs applied to time-frequency representations, 1D-CNNs process the raw or lightly processed waveform directly, requiring significantly fewer parameters and multiply-accumulate operations, which makes them naturally hardware-friendly for FPGA deployment. A 1D convolution filter slides along the time axis with a fixed kernel size, learning to

detect local waveform morphological features — such as the sharp upstroke of the QRS complex, the rounded P-wave, and the broad T-wave — at any temporal position, providing translation invariance. Compared to alternative architectures: RNNs and LSTMs process sequences step-by-step, making them fundamentally incompatible with FPGA parallelism; Transformers require millions of parameters, far exceeding the BRAM budget of the PYNQ-Z2; and traditional ML methods (SVM, KNN, Random Forest) require handcrafted feature engineering that lacks generalisation.

The proposed Hybrid-Parallel 1D-CNN departs from the conventional cascaded deep architecture (where each layer feeds the next in a sequential chain) by processing the input through multiple parallel convolutional branches simultaneously. This exploits the FPGA's physical ability to instantiate independent circuits operating concurrently on the same input data, dramatically reducing inference latency compared to a deep sequential network of equal representational capacity. The parallel branch outputs are concatenated and passed through a ReLU activation and max-pooling layer before final classification. The total trainable parameter count is approximately 25,000, matching the PACAC and YMER reference models in parameter efficiency while being substantially lighter than standard ResNet or VGG architectures.

Training Configuration

The model is built and trained in TensorFlow/Keras on a GPU-equipped development PC via Google Colab. The training configuration is as follows:

- Loss function: Categorical cross-entropy, appropriate for multi-class classification.
- Optimiser: Adam with an initial learning rate of $1e^{-3}$.
- Regularisation: Dropout (rate 0.5) applied in fully connected layers to prevent overfitting.
- Training epochs: 30, with early stopping based on validation loss.
- Batch size: 32.

The trained floating-point model is saved as a TensorFlow .h5 file, which serves as the input to the hardware synthesis stage. Model compression — INT8 quantization (FP32 to INT8, 4x memory reduction) and weight pruning (removal of near-zero weights) — is applied after training to reduce the model's arithmetic intensity and memory footprint to levels compatible with the PYNQ-Z2's on-chip resources.

3.3 Stage 3 — FPGA Accelerator Synthesis (Vitis HLS)

Vitis High-Level Synthesis (HLS) is used to convert the CNN model, described in C/C++, into synthesisable RTL hardware without requiring manual VHDL or Verilog coding. This approach dramatically accelerates development time while producing a hardware implementation that exploits the FPGA's parallel compute fabric. The CNN layers — convolution, ReLU activation, max-pooling, and fully connected — are each individually described in C++ and synthesised into configurable IP cores.

HLS optimisation pragmas (compiler directives) are applied to control the hardware architecture generated:

- **PIPELINE:** Applied to inner loop bodies, enabling the hardware to begin processing a new iteration before the previous one completes, increasing throughput by overlapping computation.
- **UNROLL:** Applied to loop bodies that operate on small, fixed-size arrays, causing the synthesiser to instantiate multiple compute units in parallel rather than executing iterations sequentially, reducing latency.

For the convolution IP core, the parallel branch architecture is preserved: both Branch A and Branch B convolution operations are implemented as independent hardware modules, ensuring they execute simultaneously on the FPGA fabric. The quantized INT8 weights and the pruned sparse weight mask are stored in the IP core's weight memory. The complete set of IP cores is exported as RTL and packaged for integration in Vivado.

3.4 Stage 4 — System Block Design and Integration (Vivado)

The Vivado IP Integrator is used to assemble the complete Zynq PS-PL co-design system. The key components are:

- **ZYNQ7 Processing System (PS):** Instantiated with the dual-core ARM Cortex-A9 processor, configured with DDR3 memory access and FCLK_CLK0 fabric clock output to drive the PL.
- **ecg_cnn_top IP Core:** The custom CNN accelerator exported from Vitis HLS, with AXI4-Lite slave interface for control and AXI4-Full master interfaces for data input and output.
- **AXI SmartConnect:** Routes AXI transactions between the PS high-performance (HP) slave

ports and the CNN IP master ports, providing address mapping and burst management.

- AXI Interconnect (Memory): Routes CNN IP output data back to DDR3 via the PS HP ports.
- AXI GPIO: Two GPIO blocks for status signalling and control.
- Processor System Reset: Manages synchronised reset distribution across all PS and PL clock domains.

The PS is linked to the PL via high-speed AXI HP ports, which provide 64-bit wide, burst-capable data paths capable of saturating the CNN accelerator's data bandwidth requirements. After placing and routing the complete design against the PYNQ-Z2's Xilinx Zynq XC7Z020 device, Vivado generates the deployment bitstream (.bit) and a hardware handoff file (.hwh) that describes the IP core address map to the PYNQ software layer.

3.5 Stage 5 — PYNQ Deployment and Real-Time Testing

The PYNQ-Z2 board runs a Ubuntu-based Linux OS from a 16 GB MicroSD card, with the PYNQ Python framework providing high-level overlay management. Deployment proceeds as follows:

- The .bit bitstream and .hwh hardware handoff files are transferred to the PYNQ-Z2 via Gigabit Ethernet.
- In a Jupyter Notebook running on the ARM PS, the overlay is loaded using the PYNQ overlay API, which programs the FPGA PL with the CNN bitstream and maps the IP core register interface into the ARM memory space.
- Test ECG beats (pre-segmented 186-sample windows) are loaded from the held-out test set into a contiguous DMA buffer in DDR3.
- The PS sends a start directive to the CNN IP core via the AXI4-Lite control interface. The IP core reads the input data from DDR3, performs parallel inference through all CNN layers, and writes the 5-class softmax output back to a result buffer.
- The PS reads the classification result and the predicted AAMI class label (N/S/V/F/Q) is extracted as the argmax of the 5-element output vector.
- Inference latency is measured as the wall-clock time from the PS sending the start signal to receiving the valid result, using Python's `time.perf_counter()` for microsecond resolution.

Total on-chip power is measured via the Vivado power analysis report, which accounts for dynamic switching power in both the PL fabric and the PS, as well as static leakage power — providing the full system power figure rather than a partial CPU-only measurement.

4. Results and Discussion

4.1 Implementation Stage Accuracy

Table 2 shows the classification accuracy of the model at each stage of the implementation pipeline. The model achieves 94.21% in the initial Google Colab software environment. This drops marginally to 94.00% after HLS synthesis, reflecting the effect of INT8 quantization on weight precision. The final deployed accuracy on the PYNQ-Z2 board is 93.20%, representing a total accuracy reduction of just 1.01 percentage points from software to hardware — a small and acceptable penalty that demonstrates the robustness of the compression and synthesis pipeline.

Table 2. Classification Accuracy Across Implementation Stages

Stage	Description	Accuracy (%)
Colab (Software)	Initial model training & evaluation on Google Colab.	94.21%
Vitis HLS (Hardware)	CNN model synthesised into HLS accelerator IP core.	94.00%
Deployed (On-Board)	Final model running on PYNQ-Z2 target board.	93.20%

4.2 FPGA Resource Utilisation

Table 3 summarises the FPGA resource utilisation on the Xilinx Zynq XC7Z020 after place-and-route in Vivado. The design is well within the resource budget of the device, with no resource type exceeding 25% utilisation. This leaves substantial headroom for future extensions — for example, adding a live ADC interface for the AD8232 ECG sensor or implementing a patient alert transmission module.

Table 3. FPGA Resource Utilisation on Xilinx Zynq XC7Z020 (PYNQ-Z2)

Resource	Used	Available	Utilisation (%)
LUT (Look-Up Tables)	10,623	53,200	19.97%
DSP Blocks	14	220	6.36%
BRAM (Block RAM)	34.5	140	24.64%
Flip-Flops (FF)	19,240	106,400	18.08%

The LUT count of 10,623 (20%) is driven primarily by the parallel convolutional branch logic. The relatively high BRAM utilisation (34.5 out of 140, or 24.6%) reflects the storage of model weights, biases, and intermediate feature maps adjacent to the compute units. The DSP usage of 14 (6.4%) is attributable to the fully connected layer multiplications; the convolution layers themselves use LUTs and shift registers rather than DSP blocks due to the small kernel sizes and INT8 fixed-point arithmetic.

4.3 Real-Time Performance Metrics

Inference Latency: The system achieves an inference latency of 60.63 ms per beat at the operational clock frequency. This comfortably satisfies the real-time requirement: at a resting heart rate of 60–100 bpm, a new R-wave peak arrives every 600–1000 ms, leaving the system 540–940 ms idle between inferences. Even at exercise heart rates of 180 bpm (R-R interval = 333 ms), the system processes each beat approximately five times faster than the next beat arrives.

Throughput: The system processes 16 beats per second, determined by $1000 \text{ ms} / 60.63 \text{ ms} \approx 16.49 \text{ beats/s}$. This exceeds the maximum clinical requirement of approximately 3–4 beats per second at extreme tachycardia rates (200 bpm), providing a comfortable safety margin.

Total On-Chip Power: The Vivado power analysis report records 1.542 W total on-chip power under active inference. This figure encompasses all dynamic switching power in the PL fabric (primarily the CNN accelerator), static leakage, and PS power. While higher than battery-operated wearable targets (typically $< 100 \text{ mW}$), the PYNQ-Z2 is a development and prototyping platform; a production implementation on a smaller process node (e.g., Zynq UltraScale+) or a dedicated ASIC would reduce power by one to two orders of magnitude.

4.4 Confusion Matrix Analysis

Table 4. Confusion Matrix — Deployed Model on PYNQ-Z2 (Test Set, AAMI EC57 5-Class)

	N (Predicted)	S (Predicted)	V (Predicted)	F (Predicted)	Q (Predicted)
Normal (N)	91	2	2	5	0
Supraventricular (S)	5	93	1	1	0
Ventricular (V)	1	1	94	4	0
Fusion (F)	3	2	5	90	0
Unknown / Paced (Q)	0	2	0	0	98

The model performs strongly across all five classes. Ventricular beats (V) achieve the highest

per-class accuracy at 94%, confirming the system’s ability to detect the most clinically critical arrhythmia class — premature ventricular contractions — with high reliability. Unknown/Paced beats (Q) achieve 98%, benefiting from their morphologically distinct waveform. Supraventricular beats (S) reach 93%, while Normal beats (N) achieve 91%. The Fusion class (F) achieves 90%, which is the weakest per-class result; this is expected, as fusion beats are morphologically intermediate between N and V beats by definition, making them inherently ambiguous. The primary off-diagonal errors for class F are misclassifications as V (5%) and N (3%), which are the two parent classes of the fusion beat — a clinically interpretable and expected failure mode.

4.5 Comparison with Reference Works

Table 5 compares the implemented system against the two PYNQ-Z2 reference works — the PACAC paper (GLSVLSI 2024) and the YMER paper (2024) — across all reported parameters. Both reference papers use the same hardware platform and toolchain, making this a direct, controlled comparison.

Table 5. Performance Comparison with Reference Works on PYNQ-Z2

Parameter	PACAC (GLSVLSI 2024)	YMER Paper (2024)	Our Work
Model Type	Hybrid 1D-CNN	Hybrid 1D-CNN	Hybrid 1D-CNN
Dataset	MIT-BIH Arrhythmia	MIT-BIH Arrhythmia	MIT-BIH Arrhythmia
Input Size	256×1 samples	256×1 samples	186×1 samples
No. of Classes	5 (N,S,V,F,Q)	5 (N,S,V,F,Q)	5 (N,S,V,F,Q)
Train / Test Split	85 / 15	85 / 15	80 / 20
Parameters	$\sim 24,865$	$\sim 23,725$	$\sim 25,000$
Accuracy	95.80%	92.72% (val)	93.20%
Latency	Not reported	Not reported	60.63 ms
Throughput	Not reported	Not reported	16 beats/s
Hardware Platform	PYNQ-Z2 FPGA	PYNQ-Z2 FPGA	PYNQ-Z2 FPGA
HLS Tool	Vitis HLS + Vivado	Vitis HLS + Vivado	Vitis HLS + Vivado
Power	Not reported	Not reported	1.542 W

The implemented system is the only one among the three to report latency (60.63 ms), throughput (16 beats/s), and total on-chip power (1.542 W), providing a complete characterisation of real-time edge deployment that the reference works do not offer. While the implemented system’s 93.20% accuracy is slightly below the PACAC paper’s 95.80%, it uses a more conservative 80/20 train-test split (vs. 85/15) and a smaller input window (186 samples vs. 256), and is the only one

to report full end-to-end deployment metrics. The accuracy difference is also consistent with the expected variance from the more challenging train-test split and the use of ENN post-processing, which may slightly reduce training set size at boundary regions.

5. Technical Challenges and Limitations

5.1 Technical Challenges Encountered

HLS Synthesis and Pipelining: Applying the PIPELINE pragma at the outermost loop level in the convolution layer caused Vitis HLS to attempt full unrolling of nested loops, resulting in extremely high synthesis times (exceeding 6 hours) and resource explosion. The issue was resolved by applying PIPELINE only at the innermost loop bodies and using UNROLL selectively at the convolution kernel loop, reducing synthesis time to under 30 minutes while preserving the parallelism benefit.

PS-PL AXI Integration: Achieving stable, deadlock-free data transfer between the ARM PS and the CNN PL IP core required multiple iterations of the Vivado block design. The AXI SmartConnect configuration needed careful tuning of burst lengths and address offsets to match the DMA buffer layout in DDR3. Misaligned memory addresses caused silent data corruption in early integration attempts, which was diagnosed by comparing PS-side and PL-side intermediate results.

Class Imbalance at Decision Boundaries: Initial experiments using SMOTE alone produced lower precision on the Supraventricular (S) and Fusion (F) classes. Analysis of misclassified samples revealed that many erroneous training instances were synthetically generated minority samples placed close to the N-class boundary. Adding ENN post-processing removed these ambiguous samples, improving S and F class performance at the cost of a slightly smaller effective training set.

Peripheral Output Integration: An initial design goal was to display classification results on the PYNQ-Z2's on-board user LEDs (LED0–LED3, one per arrhythmia class) using AXI GPIO. However, integrating the GPIO interrupt pathway into the real-time inference loop caused timing conflicts with the CNN IP core's AXI handshake, leading to intermittent missed results. The LED output was therefore replaced with laptop-side console output via the Jupyter Notebook interface, which is functionally equivalent for validation purposes.

End-to-End System Integration: The multi-stage toolchain (Python → TensorFlow → C/C++ HLS → Vivado → PYNQ) introduced multiple format conversion points, each of which required careful validation. In particular, the mapping of quantized INT8 weight tensors from the TensorFlow model to the fixed-point representation expected by the HLS IP core required a custom weight extraction and packing script.

5.2 Limitations of the Current Implementation

Dataset-Only Validation: The system is validated exclusively on pre-recorded clinical ECG data from the MIT-BIH database. No live ECG acquisition hardware (such as the AD8232 analogue front-end sensor) has been integrated. Consequently, the reported latency and accuracy figures reflect performance on a clean, pre-segmented test set, and the system has not been subjected to real-world acquisition noise, electrode artefacts, or patient motion.

Power Budget for Wearable Deployment: The 1.542 W on-chip power consumption of the PYNQ-Z2 during active inference exceeds the power budget of a typical battery-powered wearable device. The PYNQ-Z2 is a development platform, not a production chip; a production implementation on a more power-efficient fabric (e.g., Lattice Nexus, Xilinx Artix UltraScale+, or a custom ASIC) would be required for wearable deployment.

Hybrid Processing Dependency: The current architecture relies on the ARM PS for data loading, preprocessing, result formatting, and Jupyter Notebook output. This means the system is not yet a fully standalone embedded device and requires a host laptop connection for the Ethernet interface. A standalone deployment would require an embedded RTOS or bare-metal firmware on the ARM PS to replace the PYNQ Python layer.

Latency Optimisation Scope: While 60.63 ms satisfies the real-time constraint for all clinically relevant heart rates, further latency reduction is achievable. Increasing the FPGA clock frequency beyond the current operating frequency, applying data-flow optimisation across the full CNN pipeline in HLS, or restructuring the PS-PL data transfer to use zero-copy shared memory (eliminating the DMA overhead) are all viable optimisation pathways.

6. Future Scope

Live Wearable ECG Integration: The most immediate extension is to interface the PYNQ-Z2 with a real-time ECG acquisition front-end, specifically the AD8232 single-lead heart rate

monitor module, via the board's Arduino-compatible analog input pins and an on-chip ADC. The developed FPGA accelerator is designed to be sensor-ready for this transition: the AXI input bus of the CNN IP core accepts any 186-sample integer vector, regardless of its source (pre-recorded dataset or live ADC stream). Adding live acquisition would enable true end-to-end wearable validation under real physiological conditions, including motion artefacts, electrode contact variation, and patient-specific morphological variability.

Standalone Embedded System: The current implementation depends on the PYNQ Linux OS and Jupyter Notebook environment for data management and output display. Converting the system to a bare-metal or FreeRTOS application running directly on the ARM Cortex-A9, with a dedicated SPI/I2C output interface for a small OLED display or BLE transmission module, would produce a fully self-contained IoMT device suitable for unattended 24-hour Holter-style monitoring.

Patient-Specific Personalisation: The current model is trained on a population-level dataset and does not account for patient-specific ECG morphology variations. Incorporating online or periodic fine-tuning using a small buffer of patient-specific labelled beats (following the approach proposed by Kiranyaz et al.) would allow the model to adapt to individual patients, improving accuracy particularly for borderline beat types. This fine-tuning could be executed on the ARM PS while the FPGA continues real-time classification, enabling continuous model improvement without inference downtime.

Scalable Deployment in Resource-Constrained Environments: The system architecture, particularly the modular HLS IP core design, is portable across the Xilinx Zynq product family. Migration to a smaller, lower-power device (e.g., Zynq-7010 or Lattice CrossLink-NX) would reduce the bill of materials and power consumption to levels compatible with coin-cell battery operation. Conversely, deploying the accelerator on a Zynq UltraScale+ device with multiple CNN instances running in parallel would support multi-lead ECG classification for more comprehensive arrhythmia detection, including rhythm abnormalities that require multi-lead analysis (e.g., ST-elevation detection for myocardial infarction screening).

Secure Clinical Data Transmission: The current system transmits classification results over an unencrypted Ethernet connection. Integrating TLS-encrypted MQTT or BLE transmission with clinical-grade data provenance (patient ID, timestamp, device ID) would produce a system compliant with healthcare data standards such as HL7 FHIR, enabling seamless integration with

electronic health record (EHR) platforms and remote patient monitoring dashboards.

7. Conclusion

This project has successfully demonstrated the complete end-to-end design and deployment of an FPGA-based Edge AI system for real-time cardiac arrhythmia classification on the Xilinx PYNQ-Z2 development board. Starting from raw ECG signals in the MIT-BIH Arrhythmia Database, the system traverses a rigorous five-stage pipeline: clinically validated preprocessing and AAMI EC57 class mapping, Hybrid-Parallel 1D-CNN design and training with SMOTE+ENN data balancing, model compression via INT8 quantization and weight pruning, Vitis HLS synthesis of a parallelised CNN accelerator IP core with PIPELINE and UNROLL optimisations, and full PS-PL co-design integration and validation in Vivado and PYNQ.

The deployed system achieves 93.20% classification accuracy across all five AAMI EC57 arrhythmia classes — a clinically meaningful result obtained under rigorous evaluation conditions — with an inference latency of 60.63 ms per beat, a throughput of 16 beats per second, and a total on-chip power consumption of 1.542 W, using only 20% of available LUTs, 6.4% of DSPs, and 24.6% of BRAMs. These results confirm that the system satisfies the real-time constraint for continuous cardiac monitoring at all clinically relevant heart rates, while leaving substantial FPGA resources available for future sensor integration and system extensions.

Three key conclusions can be drawn from this work. First, the Hybrid-Parallel 1D-CNN architecture — replacing the conventional deep sequential model with multiple concurrent feature extraction branches — is effective at reducing inference latency while maintaining accuracy on FPGA hardware, a direct consequence of exploiting the FPGA’s physical parallelism. Second, rigorous data preparation — specifically the combined SMOTE+ENN balancing and AAMI EC57 mapping — is as important as model architecture for producing a clinically credible system; the accuracy figures of reference works that omit these steps are not comparable and inflate performance claims. Third, the PYNQ framework and Vitis HLS toolchain together provide a viable path from high-level Python model development to verified FPGA deployment without requiring deep RTL expertise, lowering the barrier to hardware acceleration in medical AI applications.

Future work will focus on live ECG acquisition via the AD8232 sensor, conversion to a standalone bare-metal firmware, and patient-specific fine-tuning to further close the gap between database

accuracy and real-world clinical performance.

References

- [1] K. P. Nandini and G. Seshikala, “Efficient ECG Arrhythmia Detection on FPGA using Machine Learning and Fiducial Windowing,” *Engineering, Technology & Applied Science Research*, vol. 15, no. 2, pp. 21100–21105, 2025. <https://doi.org/10.48084/etasr.9589>
- [2] Z. Liu, X. Ling, Y. Zhu, and N. Wang, “FPGA-based 1D-CNN accelerator for real-time arrhythmia classification,” *Journal of Real-Time Image Processing*, vol. 22, Art. no. 66, 2025. <https://doi.org/10.1007/s11554-025-01642-w>
- [3] A. L. Goldberger et al., “PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals,” *Circulation*, vol. 101, pp. e215–e220, 2000. <https://physionet.org/content/mitdb/1.0.0/>
- [4] Association for the Advancement of Medical Instrumentation (AAMI), “ANSI/AAMI EC57:2012/(R)2020 — Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms,” AAMI Standards Publication, 2020.
- [5] S. Kiranyaz, T. Ince, and M. Gabbouj, “Real-Time Patient-Specific ECG Classification by 1-D Convolutional Neural Networks,” *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016. <https://doi.org/10.1109/TBME.2015.2468589>
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-Sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. <https://doi.org/10.1613/jair.953>
- [7] Xilinx Inc., “PYNQ: Python Productivity for Zynq Platforms,” Xilinx Documentation and User Guide, 2023. <https://www.pynq.io/>
- [8] S. Mangaraj and A. Bisoyi, “PYNQ-Z2 Accelerated Cardiac Arrhythmia Classifier — A Deep Learning Based Approach,” *YMER Digital Research Publication*, 2024. <https://ymerdigital.com/uploads/YMER2404H9.pdf>